



unyts

Unit Conversion with Directed Graph Networks

Powered by Graph Search Algorithms

Version 0.10.1

March 5, 2026

<https://github.com/ayaranitram/unyts>

Contents

1	Introduction	4
1.1	The Core Innovation	4
1.2	Key Features	4
1.2.1	Pre-Loaded Unit Networks	4
1.2.2	Unit-Aware Calculations	5
1.2.3	Multiple Search Algorithms	5
1.3	Use Cases	5
1.4	Design Philosophy	6
2	The Directed Graph Foundation	7
2.1	Graph Theory Basics	7
2.1.1	What is a Directed Graph?	7
2.1.2	Graphs vs. Lookup Tables	7
2.2	unyts Implementation	8
2.2.1	The UNode Class	8
2.2.2	The Conversion Class	8
2.2.3	The UDigraph Class	9
2.2.4	Building the Network	9
2.3	Why Graphs Win	10
2.3.1	Automatic Path Discovery	10
2.3.2	Bidirectional Conversion	10
2.3.3	Composability	10
2.3.4	Extensibility	10
2.4	Summary	11
3	Search Algorithms	12
3.1	Breadth-First Search (BFS)	12
3.1.1	Algorithm Overview	12
3.1.2	Implementation	12
3.1.3	Visual Example	13
3.1.4	Use Cases	13
3.2	Depth-First Search (DFS)	13
3.2.1	Algorithm Overview	13
3.2.2	Implementation with Descendant Filtering	14
3.2.3	Visual Example	14
3.2.4	Use Cases	15
3.3	lean_BFS (Optimized BFS)	15
3.3.1	Innovation: Network Pruning	15
3.3.2	Multi-Generation Strategy	15
3.3.3	Performance Gains	16
3.3.4	Use Cases	16

3.4	hybrid_BFS (Parallel Search)	16
3.4.1	Concurrent Execution	16
3.4.2	Parallelization Options	16
3.4.3	Use Cases	17
3.5	Algorithm Comparison	17
3.5.1	Selection Guidelines	17
3.6	Summary	17
4	The Units Network	18
4.1	Network Organization Principles	18
4.1.1	Canonical Definitions	18
4.1.2	No Duplicate Paths	18
4.1.3	Modular Structure	19
4.2	Length Network	19
4.2.1	SI Prefix Structure	19
4.2.2	Imperial Chain Structure	19
4.2.3	SI-Imperial Bridge	19
4.2.4	Complete Length Network with Highlighted Path	19
4.3	Time Network	20
4.4	Temperature Network	20
4.5	Other Network Segments	20
4.5.1	Volume Network	20
4.5.2	Pressure Network	21
4.5.3	Mass Network	21
4.6	Example Conversion Walkthrough	21
4.6.1	Step 1: Find Path	21
4.6.2	Step 2: Apply Transformations	22
4.6.3	Step 3: Cache Result	22
4.7	Summary	22
5	Installation	23
5.1	Requirements	23
5.1.1	Python Version	23
5.1.2	Core Dependencies	23
5.1.3	Optional Dependencies	23
5.2	Installation Methods	23
5.2.1	Install from PyPI (Recommended)	23
5.2.2	Upgrade to Latest Version	24
5.2.3	Install from Source	24
5.3	Verification	24
5.4	First-Time Setup	24
5.5	Configuration Files	24
5.6	Troubleshooting Installation	25
5.6.1	Missing cloudpickle	25
5.6.2	Import Errors	25
5.6.3	Version Conflicts	25
6	Quick Start	26
6.1	Basic Conversion	26
6.2	Creating Unit-Aware Quantities	26
6.3	Type Checking	27
6.4	Converting Unit Instances	27

6.5	Working with Arrays	27
6.6	Controlling Conversion Path Display	27
6.7	Changing the Search Algorithm	28
7	Graphical User Interface	29
7.1	Main Window	29
7.2	Bidirectional Conversion	30
7.3	File Menu	30
7.3.1	Conversions Memory	30
7.3.2	Delete Cache Files	30
7.4	Options Menu	30
7.4.1	Set FVF (Formation Volume Factor)	30
7.4.2	Set Density	31
7.4.3	General Options	31
7.4.4	Search Algorithm	31
7.4.5	Parallel	32
7.5	Help Menu	32
7.6	Conversion Path Display	32
7.7	Error Handling in GUI	33
8	Python API Reference	34
8.1	Core Functions	34
8.1.1	convert()	34
8.1.2	units()	35
8.2	Unit Class	35
8.2.1	Attributes	35
8.2.2	Arithmetic Operations	36
8.2.3	Comparison Operators	37
8.2.4	.to() Method	37
8.3	Module-Level Functions	37
8.3.1	Configuration	37
8.3.2	Memory Management	38
8.3.3	Path Inspection	38
8.3.4	Custom Units and Conversions	38
8.3.5	Formation Volume Factor (FVF)	38
8.3.6	Constant Density	39
8.4	Type Checking and Subclasses	39
8.5	NumPy and Pandas Integration	39
8.6	Uncertain Units	40
9	Troubleshooting	41
9.1	Installation Issues	41
9.1.1	Module Not Found	41
9.1.2	Dependency Conflicts	41
9.2	Conversion Errors	41
9.2.1	NoConversionFoundError	41
9.2.2	SearchTimeoutError	42
9.2.3	NoFVFError	42
9.2.4	WrongFormatError	43
9.3	GUI Issues	43
9.3.1	GUI Won't Launch	43
9.3.2	GUI Crashes on Conversion	43

9.4	Performance Issues	43
9.4.1	Slow First Conversion	43
9.4.2	Slow Subsequent Conversions	44
9.5	Cache Corruption	44
9.6	Logging and Debugging	44
9.6.1	Enable Verbose Mode	44
9.6.2	Adjust Logging Level	44
9.6.3	Inspect Network Structure	45
9.7	Getting Help	45

Chapter 1

Introduction

unyts is an innovative Python package that revolutionizes unit conversion and unit-aware calculations by leveraging directed graph network theory combined with graph search algorithms.

1.1 The Core Innovation

Traditional unit converters rely on exhaustive lookup tables containing pre-computed conversion factors for every possible unit pair. This approach suffers from several limitations:

- Scalability: Adding new units requires defining conversions to all existing units
- Maintenance: Each new unit adds $O(n)$ conversion definitions
- Compound units: Ratios like **kg/m³** or **stb/day/psi** explode the lookup table size
- Memory overhead: Storing redundant conversion paths

The **unyt**s package takes a fundamentally different approach by modeling units and conversions as a directed graph (digraph):

- Each unit is a node in the graph
- Each conversion rule is a directed edge with a transformation function
- Breadth-First Search (BFS) discovers conversion paths automatically
- Units are defined by their official definitions (e.g., 1 foot = 12 inches)

This graph-based architecture enables conversion between any two compatible units without requiring explicit definition of every conversion pair. The search algorithms find the optimal path through the network automatically.

1.2 Key Features

1.2.1 Pre-Loaded Unit Networks

The package ships with comprehensive networks covering:

- Length: SI (meter with prefixes) and Imperial (inch, foot, yard, mile, etc.)
- Area: square units for all length systems

- Volume: cubic units, gallons (US & UK), liters, barrels
- Mass/Weight: metric (gram, kilogram) and Imperial (pound, ounce, ton)
- Time: millisecond through century
- Temperature: Celsius, Fahrenheit, Kelvin, Rankine
- Pressure: Pascal, bar, psi (gauge & absolute)
- Energy, Power, Density, Viscosity, Data, and more

1.2.2 Unit-Aware Calculations

The **Unit** class hierarchy provides:

- Arithmetic operations: Addition, subtraction, multiplication, division with automatic unit handling
- Logical comparisons: Units automatically convert before comparison
- Exponentiation: Raising units to powers (e.g., m^2 , ft^3)
- Compound unit creation: Multiplying $m \times m$ yields m^2

1.2.3 Multiple Search Algorithms

Four specialized algorithms optimize different scenarios:

- BFS: Guarantees shortest path, layer-by-layer exploration
- lean_BFS: Optimized BFS with pre-filtered network (default)
- DFS: Fast depth-first search with descendant filtering
- hybrid_BFS: Parallel execution of BFS and lean_BFS

1.3 Use Cases

unyts excels in:

- Scientific computing: Heterogeneous unit handling in calculations
- Engineering applications: Converting between SI and Imperial systems
- Oil & gas industry: Specialized units (API gravity, standard barrels, formation volume factors)
- Data analysis: pandas/NumPy integration for unit-aware arrays
- Education: Teaching dimensional analysis and unit systems

1.4 Design Philosophy

1. Canonical definitions: Units defined by international standards (1 yard = 0.9144 meters)
2. No duplicate paths: Each unit pair connected by single canonical route
3. Composability: Complex ratios built from simple units
4. Extensibility: Add custom units without modifying existing network
5. Performance: Search memory caching for repeated conversions

The following chapters explore the technical foundations (graph theory, search algorithms), provide complete installation and usage guides, and document the full Python API.

Chapter 2

The Directed Graph Foundation

This chapter explains the graph-theoretic foundations of **unyts**, demonstrating why directed graphs provide superior unit conversion compared to traditional lookup tables.

2.1 Graph Theory Basics

2.1.1 What is a Directed Graph?

A directed graph (digraph) $G = (V, E)$ consists of:

- A set of vertices (nodes) $V = \{v_1, v_2, \dots, v_n\}$
- A set of directed edges $E \subseteq V \times V$
- Each edge $e = (v_i, v_j)$ points from source v_i to target v_j

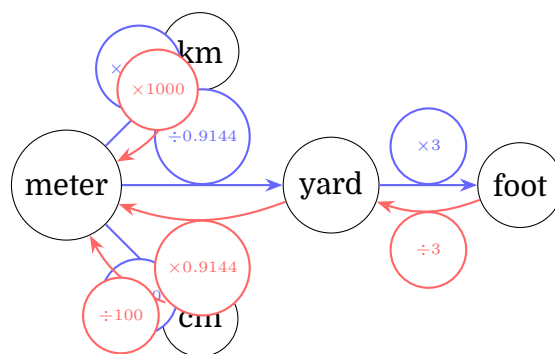


Figure 2.1: Simple directed graph showing length unit conversions. Blue edges represent forward conversions, red edges represent reverse conversions.

In Figure 2.1, each node represents a unit, and directed edges encode conversion factors. Notice that edges are bidirectional—both forward and reverse conversions exist.

2.1.2 Graphs vs. Lookup Tables

The key advantage: adding a new unit to a graph requires connecting it to one or two existing units (e.g., connecting a new length unit only to **meter**), whereas a lookup table requires defining conversions to all existing units.

Feature	Lookup Table	Graph Network
Add 1 unit	Define n conversions	Define 1–2 edges
Storage	$O(n^2)$	$O(n)$ or $O(n \log n)$
Query time	$O(1)$	$O(n)$ with caching $\rightarrow O(1)$
Compound units	Exponential explosion	Compositional
Maintenance	Duplicate definitions	Canonical paths
Extensibility	Rigid	Dynamic

Table 2.1: Comparison of lookup tables vs. graph-based conversion

2.2 unyts Implementation

2.2.1 The UNode Class

Each unit in the network is represented by a `UNode` object:

```

1 class UNode:
2     __slots__ = ('name',)
3
4     def __init__(self, name: str):
5         self.name = name
6
7     def __eq__(self, other):
8         return self.name == other.name
9
10    def __hash__(self):
11        return hash(self.name)

```

Listing 2.1: UNode implementation (simplified)

Key design choices:

- Equality based on name: Nodes created in different sessions compare equal
- Hashable: Enables use as dictionary keys for search memory caching
- Lightweight: Minimal memory footprint with `__slots__`

2.2.2 The Conversion Class

Directed edges encode transformation functions:

```

1 class Conversion:
2     def __init__(self, source: UNode, target: UNode,
3                 function: callable):
4         self.source = source
5         self.target = target
6         self.function = function

```

Listing 2.2: Conversion class structure

A conversion from meters to kilometers stores:

```

1 meter_to_km = Conversion(
2     source=UNode('meter'),
3     target=UNode('kilometer'),
4     function=lambda x: x * 0.001
5 )

```

2.2.3 The UDigraph Class

The master graph object manages the entire network:

```

1 class UDigraph:
2     def __init__(self):
3         self.edges = {} # {UNode: ([children], [conversions])}
4         self.memory = {} # Cached search results
5         self.fvf = None # Formation volume factor
6
7     def add_node(self, node: UNode):
8         if node not in self.edges:
9             self.edges[node] = ([], [])
10
11    def add_edge(self, conversion: Conversion):
12        source = conversion.source
13        if source not in self.edges:
14            self.add_node(source)
15        children, conversions = self.edges[source]
16        children.append(conversion.target)
17        conversions.append(conversion.function)
18
19    def children_of(self, node: UNode):
20        return self.edges[node][0] if node in self.edges else []

```

Listing 2.3: UDigraph core attributes

The `edges` dictionary maps each node to a tuple of (`child_nodes`, `conversion_functions`), enabling efficient traversal during search.

2.2.4 Building the Network

Unit conversions are defined based on official standards:

```

1 network = UDigraph()
2
3 # Define nodes
4 meter = UNode('meter')
5 kilometer = UNode('kilometer')
6 yard = UNode('yard')
7 foot = UNode('foot')
8
9 network.add_node(meter)
10 network.add_node(kilometer)
11 network.add_node(yard)
12 network.add_node(foot)
13
14 # Define conversions (both directions)
15 network.add_edge(Conversion(meter, kilometer,
16                             lambda x: x * 0.001))
17 network.add_edge(Conversion(kilometer, meter,
18                             lambda x: x * 1000))
19 network.add_edge(Conversion(meter, yard,
20                             lambda x: x / 0.9144))
21 network.add_edge(Conversion(yard, meter,
22                             lambda x: x * 0.9144))
23 network.add_edge(Conversion(yard, foot,
24                             lambda x: x * 3))
25 network.add_edge(Conversion(foot, yard,

```

```
26 lambda x: x / 3))
```

Listing 2.4: Example network construction

2.3 Why Graphs Win

2.3.1 Automatic Path Discovery

Once the network is built, conversions between any compatible unit pair are automatic. To convert 100 centimeters to feet:

1. BFS finds path: **cm** → **m** → **yard** → **foot**
2. Apply transformations: $100 \div 100 \times (1/0.9144) \times 3 = 3.281$ feet
3. Cache result for future use

No explicit **cm** → **foot** conversion was ever defined!

2.3.2 Bidirectional Conversion

Every conversion automatically works in both directions. Defining **meter** → **yard** and its inverse enables conversions in either direction plus all transitive paths (meter → yard → foot).

2.3.3 Composability

Compound units emerge naturally:

- **kg/m3** decomposes to **kg** and **m3**
- Each component converts independently
- **stb/day/psi** = (standard barrels) / (day) / (pounds per square inch)

The graph handles numerator and denominator units separately, then recombines.

2.3.4 Extensibility

Adding industry-specific units requires minimal changes:

```
1 from unyts import set_unit, set_conversion
2
3 # Add a custom unit
4 set_unit('my_unit')
5
6 # Define conversion to an existing unit
7 set_conversion('my_unit', 'meter',
8               lambda x: x * 42.0,
9               lambda x: x / 42.0)
```

Listing 2.5: Adding custom units

The new unit immediately participates in all graph search operations, enabling conversions to any compatible unit without additional definitions.

2.4 Summary

The directed graph architecture provides:

1. Scalability: $O(n)$ storage vs. $O(n^2)$ lookup tables
2. Maintainability: Canonical definitions prevent inconsistencies
3. Flexibility: Automatic discovery of conversion paths
4. Composability: Natural handling of compound units
5. Extensibility: Add units without modifying existing network

The next chapter explores the graph search algorithms that traverse this network efficiently.

Chapter 3

Search Algorithms

unytS implements four specialized graph search algorithms, each optimized for different conversion scenarios. This chapter explains their operation, performance characteristics, and use cases.

3.1 Breadth-First Search (BFS)

3.1.1 Algorithm Overview

BFS explores the graph layer by layer, visiting all nodes at distance d before exploring nodes at distance $d + 1$. This guarantees finding the shortest path between two nodes.

Key properties:

- Completeness: Finds a path if one exists
- Optimality: Always returns the shortest path
- Time complexity: $O(|V| + |E|)$ where V = nodes, E = edges
- Space complexity: $O(|V|)$ for the queue and visited set

3.1.2 Implementation

```
1 def BFS(graph, start, end):
2     path_queue = [[start]]      # Queue of paths
3     visited = []                # Visited paths
4
5     while path_queue:
6         current_path = path_queue.pop(0) # Dequeue
7         last_node = current_path[-1]
8
9         if current_path in visited:
10             continue
11         visited.append(current_path)
12
13         if last_node == end:
14             return current_path # Found!
15
16         # Expand to children
17         for child in graph.children_of(last_node):
18             if child not in current_path:
19                 new_path = current_path + [child]
```

```

20     path_queue.append(new_path)
21
22     return None # No path found

```

Listing 3.1: BFS pseudocode

3.1.3 Visual Example

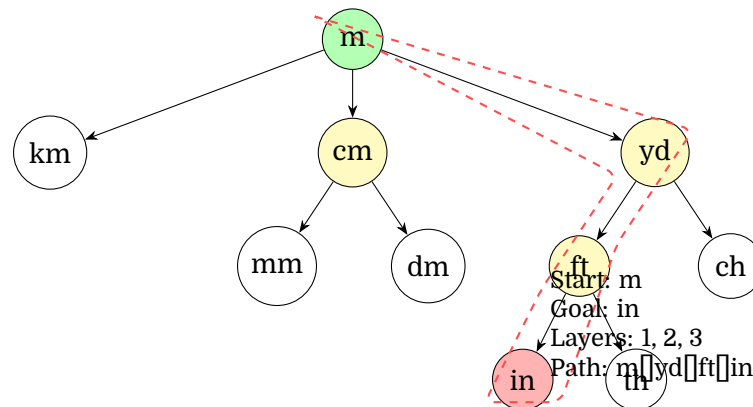


Figure 3.1: BFS exploration tree. Yellow nodes explored before red goal node (in). Green node is start. Dashed line shows shortest path found.

In Figure 3.1, BFS explores level 1 (km, cm, yd), then level 2 (mm, dm, ft, ch), discovering the shortest 3-hop path: meter → yard → foot → inch.

3.1.4 Use Cases

- When shortest conversion path is required
- Unknown network topology
- Verification of conversion accuracy
- Teaching/debugging conversion logic

3.2 Depth-First Search (DFS)

3.2.1 Algorithm Overview

DFS explores as deep as possible along each branch before backtracking. It uses descendant filtering to avoid exploring branches that cannot reach the goal.

Key properties:

- Completeness: Finds a path (but not necessarily shortest)
- Optimality: No guarantee of shortest path
- Time complexity: $O(|V| + |E|)$ with pruning
- Space complexity: $O(|V|)$ for recursion stack
- Advantage: Often faster for deep paths

3.2.2 Implementation with Descendant Filtering

The `unyt`s DFS implementation adds intelligent pruning:

```

1 def DFS(graph, start, end, branch_depth=25):
2     def dfs_recursive(node, visited, path):
3         visited.add(node)
4
5         for child in graph.children_of(node):
6             if child in visited:
7                 continue
8
9             # Pruning: skip if goal unreachable within depth
10            if end not in get_descendants(child, branch_depth):
11                continue
12
13            new_path = path + [child]
14
15            if child == end:
16                return new_path # Found!
17
18            result = dfs_recursive(child, visited, new_path)
19            if end in result:
20                return result
21
22        return path
23
24    return dfs_recursive(start, set(), [start])

```

Listing 3.2: DFS with descendant filtering

The `get_descendants(node, depth)` function precomputes all nodes reachable within `depth` hops, preventing exploration of dead-end branches.

3.2.3 Visual Example

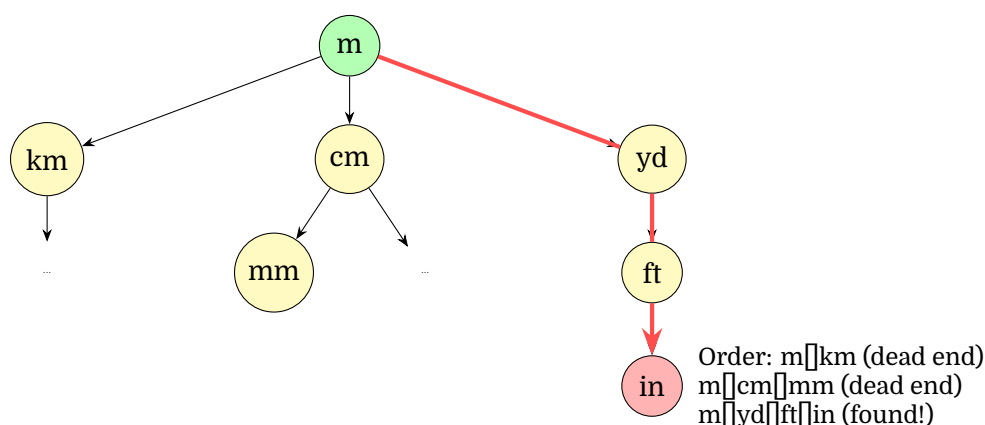


Figure 3.2: DFS exploration. Thick red arrows show the path DFS follows. Yellow nodes explored before finding goal.

DFS may explore fewer nodes than BFS if the goal happens to be on the first deep branch explored (though it might also explore more if unlucky).

3.2.4 Use Cases

- Known deep conversion paths (e.g., complex compound units)
- Memory-constrained environments
- When first valid path is acceptable

3.3 lean_BFS (Optimized BFS)

3.3.1 Innovation: Network Pruning

The classic BFS bottleneck is exploring nodes irrelevant to the start→end conversion. lean_BFS addresses this by:

1. Compute descendants of **start** up to depth d : D_{start}
2. Compute descendants of **end** up to depth d : D_{end}
3. Find intersection: $D_{relevant} = D_{start} \cap D_{end}$
4. Construct slimmed network containing only nodes in $D_{relevant}$
5. Run standard BFS on this smaller network

3.3.2 Multi-Generation Strategy

lean_BFS iteratively tries increasing depths: [0, 1, 2, 3, 4, 5, 7, 10, 13, 16, 20, 25, ...] until the intersection is non-empty. This avoids over-expanding for simple conversions while handling complex ones.

```

1 def lean_BFS(graph, start, end, max_gen=25):
2     generations = [0, 1, 2, 3, 4, 5, 7, 10, 13, 16, 20, 25]
3
4     for depth in generations:
5         if depth > max_gen:
6             break
7
8         start_desc = get_descendants(start, depth)
9         end_desc = get_descendants(end, depth)
10        intersection = start_desc & end_desc
11
12        if intersection:
13            # Build slim network with only these nodes
14            slim_edges = {
15                node: graph.edges[node]
16                for node in intersection
17                if node in graph.edges
18            }
19            slim_graph = SlimUDigraph(slim_edges)
20            return BFS(slim_graph, start, end)
21
22    return None # No path found

```

Listing 3.3: lean_BFS network filtering

3.3.3 Performance Gains

Typical performance improvement over standard BFS:

- Network size reduction: From 500+ nodes to 10-30 relevant nodes
- Speed improvement: 5-20× faster for typical conversions
- Memory savings: Proportional to network reduction

3.3.4 Use Cases

- Default algorithm for most conversions
- Production systems requiring fast response
- Repeated conversions between popular unit pairs

3.4 hybrid_BFS (Parallel Search)

3.4.1 Concurrent Execution

hybrid_BFS runs BFS and lean_BFS simultaneously in separate threads (or processes), returning the first successful result:

```
1 def hybrid_BFS(graph, start, end):
2     results = {'bfs': '', 'lean_bfs': ''}
3
4     thread_bfs = Thread(target=run_bfs,
5                          args=(results, graph, start, end))
6     thread_lean = Thread(target=run_lean_bfs,
7                          args=(results, graph, start, end))
8
9     thread_bfs.start()
10    thread_lean.start()
11
12    while True:
13        if results['lean_bfs']:
14            thread_bfs.terminate()
15            return results['lean_bfs']
16        elif results['bfs']:
17            thread_lean.terminate()
18            return results['bfs']
19        elif results['bfs'] is None and results['lean_bfs'] is None:
20            return None # Both failed
```

Listing 3.4: hybrid_BFS parallel execution

3.4.2 Parallelization Options

unyt supports multiple parallel backends:

- Threading: Default for Python <3.13 (GIL-limited)
- Multiprocessing: Python 3.14+ with free-threading (true parallelism)
- Auto: Automatically selects best available method

3.4.3 Use Cases

- Unknown path complexity
- Time-critical conversions
- Multi-core systems
- When willing to trade CPU for speed

3.5 Algorithm Comparison

Algorithm	Shortest Path	Speed	Memory	Best For
BFS	Yes	Medium	High	Shortest guarantee
DFS	No	Fast (deep)	Low	Known deep paths
lean_BFS	Yes	Fast	Medium	Most conversions
hybrid_BFS	Yes	Fastest	Highest	Critical speed

Table 3.1: Search algorithm comparison

3.5.1 Selection Guidelines

- Default: Use **lean_BFS** (fast + optimal)
- Verification: Use **BFS** to confirm shortest path
- Complex units: Try **DFS** for deep compound ratios
- Maximum speed: Use **hybrid_BFS** with multiprocessing

The algorithm can be changed globally or per-conversion:

```
1 import unyts
2
3 # Set globally
4 unyts.set_algorithm('hybrid_BFS')
5
6 # All conversions use hybrid_BFS from now on
7 result = convert(100, 'cm', 'in')
```

3.6 Summary

unyts provides four specialized search algorithms, each optimized for different scenarios. The default **lean_BFS** combines the shortest-path guarantee of BFS with network pruning for excellent performance. Users can select algorithms based on their specific needs: verification (BFS), deep paths (DFS), or maximum speed (hybrid_BFS).

Chapter 4

The Units Network

This chapter explains how the unit conversion network is organized, how conversions are defined based on official standards, and visualizes key network segments.

4.1 Network Organization Principles

4.1.1 Canonical Definitions

Every unit conversion in **unyt**s is based on official international standards or widely accepted definitions:

- 1 foot = 12 inches (by definition)
- 1 yard = 3 feet (by definition)
- 1 yard = 0.9144 meters (exact, 1959 international yard)
- 1 kilometer = 1000 meters (SI prefix definition)
- 1 pound = 0.45359237 kilograms (exact, by definition)

This ensures consistency and prevents accumulation of rounding errors across multiple conversions.

4.1.2 No Duplicate Paths

A key design principle: between any two units, there exists exactly one canonical conversion path. This prevents:

- Conflicting conversion factors
- Cyclic dependencies with rounding errors
- Ambiguity in path selection

For example, to convert **foot** to **meter**, the path is uniquely:

foot → yard → meter

There is no direct **foot** → **meter** edge, ensuring all foot-to-meter conversions use the same computation.

4.1.3 Modular Structure

The network is organized into loosely-coupled modules:

1. SI System: Each base unit (meter, gram, second) connects to its SI prefixes
2. Imperial System: Units form chains based on their definitions
3. Bridges: Connect SI and Imperial systems at specific units
4. Specialized Systems: Temperature, pressure gauges, oil field units

4.2 Length Network

4.2.1 SI Prefix Structure

The meter serves as the SI base unit for length, with prefix multipliers defined by powers of 10:

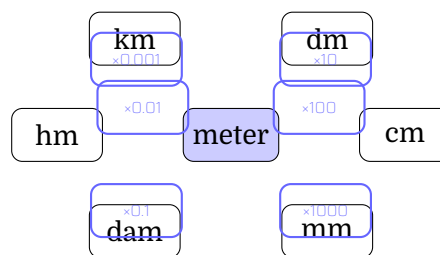


Figure 4.1: SI prefix network for length. All conversions route through the base unit (meter).

Additional SI prefixes (micro, nano, pico, kilo, mega, giga, etc.) follow the same pattern. The meter acts as a hub node for the SI subsystem.

4.2.2 Imperial Chain Structure

Imperial units form a linear chain based on their historical definitions:

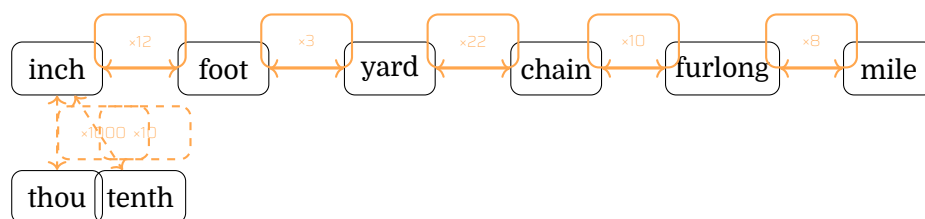


Figure 4.2: Imperial length chain. Units connected sequentially by their definitions.

4.2.3 SI-Imperial Bridge

The two systems connect at a single point: yard ↔ meter

4.2.4 Complete Length Network with Highlighted Path

Figure 4.4 shows the complete length network with a highlighted conversion path from centimeter to inch:

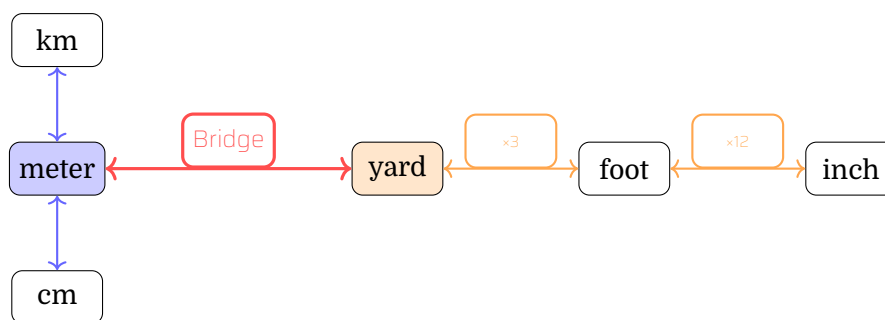


Figure 4.3: The SI-Imperial bridge at meter↔yard. All conversions between systems must route through this connection.

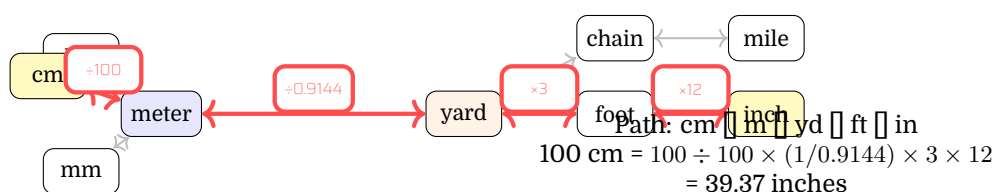


Figure 4.4: Complete length network showing highlighted conversion path from centimeter to inch (yellow nodes). Red thick arrows trace the conversion route.

4.3 Time Network

The time network forms a hierarchical tree structure:

Note that **month** uses the average month length (30.4375 days) and conversions involving months are approximate. The year node connects to **day** (365.25 days) and **month** (12 months), providing both paths.

4.4 Temperature Network

Temperature conversions are non-linear due to offset + scale transformations:

The conversion functions for temperature include both multiplication and addition:

```
1 Celsius_to_Fahrenheit = lambda C: C * 9/5 + 32
2 Fahrenheit_to_Celsius = lambda F: (F - 32) * 5/9
```

Listing 4.1: Temperature conversion example

4.5 Other Network Segments

4.5.1 Volume Network

Volume conversions split into three subsystems:

- Cubic units: m^3 , cm^3 , ft^3 , in^3 (derived from length)
- Liquid volume (US): gallon → quart → pint → gill → fluid ounce
- Liquid volume (UK): Imperial gallon → quart → pint (different definitions than US)
- Bridge: Liter ↔ cm^3 (exact), gallon(UK) ↔ liter, gallon(US) ↔ cubic inch

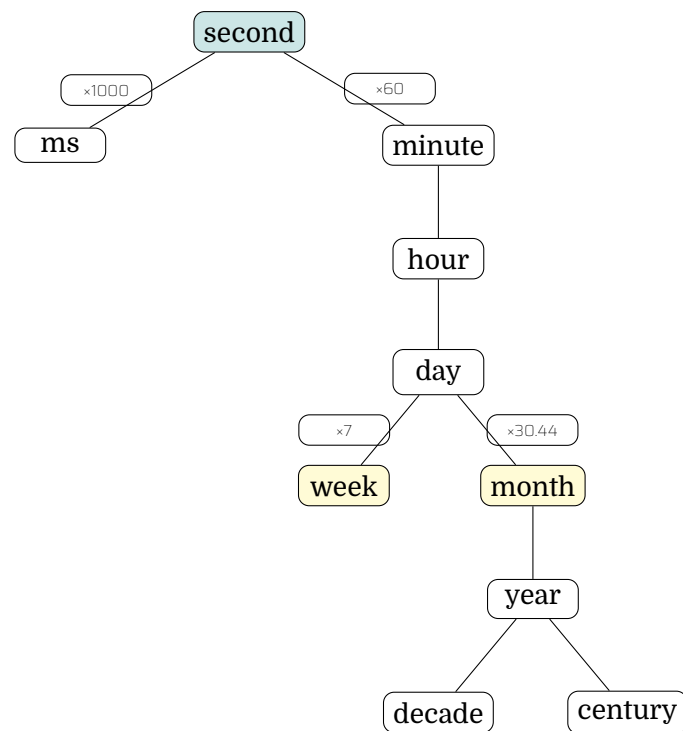


Figure 4.5: Time network hierarchical structure. Yellow nodes (week, month) branch from day.

4.5.2 Pressure Network

Pressure includes both absolute and gauge measurements:

- Absolute: Pascal (Pa), bar, psi(a), atmosphere, Torr
- Gauge: barg ↔ bara (offset by atmospheric pressure), psig ↔ psia
- Conversions: bar ↔ Pascal ↔ atmosphere, psi ↔ bar

4.5.3 Mass Network

Similar to length, mass has SI (gram-based) and Imperial (pound-based) subsystems:

- SI: kilogram → gram (with prefixes: mg, kg, tonne)
- Imperial: pound → ounce → dram, ton → hundredweight → stone → quarter
- Bridge: pound ↔ kilogram (exact conversion)

4.6 Example Conversion Walkthrough

Let's trace the conversion of 100 centimeters to inches step by step:

4.6.1 Step 1: Find Path

lean_BFS searches and finds:

cm → m → yard → foot → inch

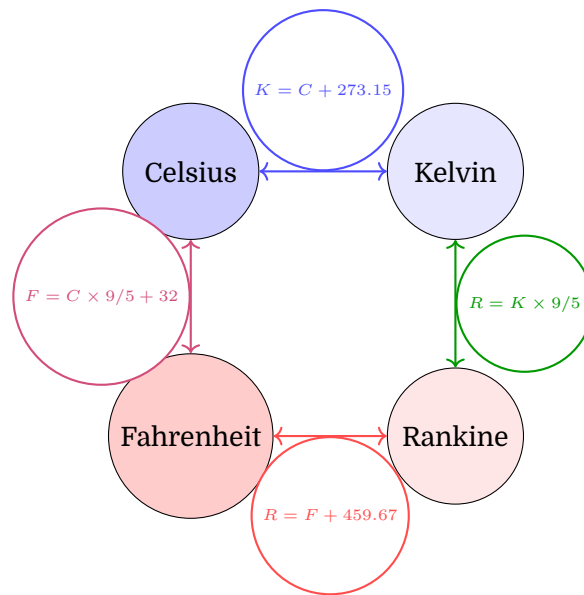


Figure 4.6: Temperature network showing all bidirectional conversions. Each edge encodes both offset and scale transformations.

4.6.2 Step 2: Apply Transformations

cm → m :	$100 \div 100 = 1.0$ meter
m → yard :	$1.0 \div 0.9144 = 1.0936$ yards
yard → foot :	$1.0936 \times 3 = 3.2808$ feet
foot → inch :	$3.2808 \times 12 = 39.37$ inches

4.6.3 Step 3: Cache Result

The conversion path and final multiplier ($39.37/100 = 0.3937$) are cached:

```

1 units_network.memory[('cm', 'inch')] = {
2     'path': ['cm', 'm', 'yard', 'foot', 'inch'],
3     'factor': 0.3937
4 }
```

Future **cm** → **inch** conversions retrieve this cached result instantly ($O(1)$ lookup).

4.7 Summary

The **units** network is carefully organized around:

1. Official definitions: All conversions based on international standards
2. Single canonical paths: No duplicate routes between unit pairs
3. Modular structure: SI, Imperial, and specialized subsystems
4. Strategic bridges: Minimal connections between subsystems
5. Hub topology: Base units (meter, gram, second) serve as central nodes

This architecture enables automatic discovery of conversion paths for thousands of unit combinations from a compact set of defining relationships.

Chapter 5

Installation

5.1 Requirements

5.1.1 Python Version

unyts supports Python 3.7 and later. The package metadata targets:

```
requires-python = ">=3.7"
```

Recommended: Python 3.10+ for optimal performance.

5.1.2 Core Dependencies

Required packages (automatically installed with **unyts**):

- **stringthings**: Helper utilities for string processing
- **cloudpickle**: Caching network and search memory

5.1.3 Optional Dependencies

While not required, the following packages enhance **unyts** functionality:

- NumPy: Enables array-aware conversions, handles lists/ndarrays
- Pandas: Recognizes Series/DataFrame objects for bulk conversions
- openpyxl: Required only for exporting the network to Excel format

Install optional dependencies manually if needed:

```
pip install numpy pandas openpyxl
```

5.2 Installation Methods

5.2.1 Install from PyPI (Recommended)

The simplest installation method:

```
pip install unyts
```

5.2.2 Upgrade to Latest Version

To ensure you have the latest features and bug fixes:

```
pip install --upgrade unyts
```

5.2.3 Install from Source

For development or testing unreleased features:

```
git clone https://github.com/ayaranitram/unyts.git
cd unyts
pip install -e .
```

The `-e` flag installs in editable mode, allowing you to modify source code and see changes immediately.

5.3 Verification

Verify the installation by checking the version:

```
1 import unyts
2 print(unyts.__version__) # Should print: 0.10.1 (or later)
```

Test basic functionality:

```
1 from unyts import convert
2
3 result = convert(1, 'meter', 'foot')
4 print(result) # Should print: 3.280839895013123
```

If no errors occur, `unyts` is correctly installed and operational.

5.4 First-Time Setup

On first import, `unyts` performs one-time initialization:

1. Builds the unit conversion network from definitions
2. Loads search memory cache (if available)
3. Creates user configuration directory

This process may take a few seconds. Subsequent imports are faster due to caching.

5.5 Configuration Files

`unyts` stores configuration and cache files in a user-specific directory:

- Windows: `C:\Users\<username>\.unyts\`
- macOS/Linux: `~/.unyts/`

Files stored:

- `parameters.ini`: User preferences (algorithm, print path, etc.)

- `search_memory.cache`: Cached conversion paths
 - `units_network.cache`: Serialized network (future optimization)
- To reset `unyts` to defaults, delete this directory and re-import.

5.6 Troubleshooting Installation

5.6.1 Missing cloudpickle

If you see warnings about cloudpickle:

```
Warning: Missing `cloudpickle` package.  
Not able to cache search memory.
```

Install it explicitly:

```
pip install cloudpickle
```

5.6.2 Import Errors

If imports fail with `ModuleNotFoundError`, ensure `unyts` is installed in the active Python environment:

```
pip list | grep unyts
```

If using virtual environments or conda, activate the correct environment before installation.

5.6.3 Version Conflicts

If upgrading causes issues, completely remove the old version first:

```
pip uninstall unyts  
pip install unyts
```

Chapter 6

Quick Start

This chapter provides a concise introduction to using **unyts** for common tasks.

6.1 Basic Conversion

The simplest way to use **unyts** is via the **convert()** function:

```
1 from unyts import convert
2
3 # Convert 10 feet to meters
4 meters = convert(10, 'ft', 'm')
5 print(meters) # Output: 3.048
```

The function signature is:

```
1 convert(value, from_units, to_units,
2         print_conversion_path=None, use_cache=True)
```

- **value**: Numeric input (int, float, list, np.array, pd.Series)
- **from_units**: Source unit string (e.g., 'ft', 'kg/m3')
- **to_units**: Target unit string
- **print_conversion_path**: Override global print_path setting
- **use_cache**: Set **False** to force fresh search

6.2 Creating Unit-Aware Quantities

Use the **units()** factory function to create **Unit** instances:

```
1 from unyts import units
2
3 q = units(6, 'in') + units(1, 'ft')
4 print(q) # Output: 18_in
```

Arithmetic operations automatically handle unit conversions:

```
1 length = units(5, 'm')
2 width = units(200, 'cm')
3 area = length * width
4 print(area) # Output: 10.0_m2
```

6.3 Type Checking

Check if a variable is unit-aware using `isinstance()`:

```
1 from unyts import Unit, units
2
3 value = units(5, 'm')
4 print(isinstance(value, Unit)) # Output: True
```

Important: Use `isinstance()`, not `type() is Unit`, because `units()` returns subclasses (e.g., `Length`, `Volume`).

6.4 Converting Unit Instances

Use the `.to()` method to convert instances:

```
1 length = units(100, 'cm')
2 print(length.to('in')) # Output: 39.37_in
3 print(length.to('ft')) # Output: 3.281_ft
```

6.5 Working with Arrays

`unyts` supports NumPy arrays and lists:

```
1 import numpy as np
2 from unyts import convert, units
3
4 # Convert list
5 temps_C = [0, 10, 20, 30]
6 temps_F = convert(temps_C, 'Celsius', 'Fahrenheit')
7 print(temps_F) # [32.0, 50.0, 68.0, 86.0]
8
9 # Unit-aware array
10 distances = units(np.array([1, 2, 3, 4, 5]), 'km')
11 print(distances.to('mi'))
12 # Output: [0.621 1.243 1.864 2.485 3.107]_mi
```

6.6 Controlling Conversion Path Display

By default, `unyts` prints the conversion path on first use:

```
1 from unyts import convert
2 convert(1, 'km', 'mi')
3 # Output: km > m > yard > foot > inch > ... > mile
4 #          0.6213711922373339
```

Toggle this behavior globally:

```
1 import unyts
2
3 unyts.print_path(False) # Disable
4 unyts.print_path(True)  # Enable
5 unyts.print_path()      # Toggle current state
```

Or control per-conversion:

```
1 convert(1, 'km', 'mi', print_conversion_path=True)
```

6.7 Changing the Search Algorithm

Switch algorithms globally:

```
1 import unyts
2
3 unyts.set_algorithm('BFS')           # Standard BFS
4 unyts.set_algorithm('lean_BFS')     # Optimized (default)
5 unyts.set_algorithm('DFS')          # Depth-first
6 unyts.set_algorithm('hybrid_BFS')   # Parallel
```

See Chapter 3 for algorithm details and selection guidelines.

Chapter 7

Graphical User Interface

The Unyts GUI provides an intuitive visual interface for unit conversions. Launch it from Python:

```
1 import unyts
2 unyts.start_gui()
```

Or from the command line:

```
1 python -m unyts
```

7.1 Main Window

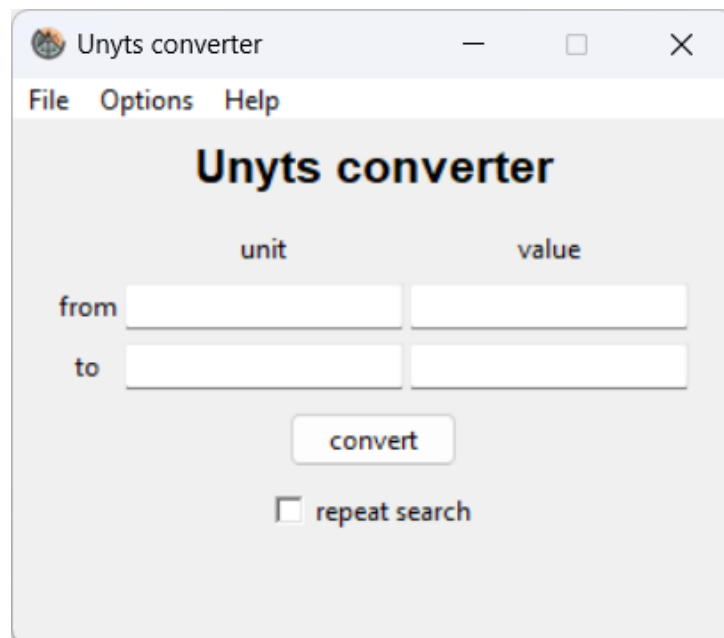


Figure 7.1: Unyts GUI main window showing conversion interface

The main window (Figure 7.1) features:

- From unit and From value: Source unit and quantity
- To unit and To value: Target unit and result

- Convert button: Execute conversion (or press Enter in any field)
- Repeat search checkbox: Force re-search (ignores cache)
- Conversion path display: Shows graph traversal path when enabled

7.2 Bidirectional Conversion

The GUI supports conversions in both directions:

- Forward: Enter value and unit in "from" fields, specify target "to" unit → press Enter or click Convert
- Reverse: Enter desired value in "to" field → calculates required "from" value

This bidirectional flexibility eliminates the need to invert calculations manually.

7.3 File Menu

7.3.1 Conversions Memory

The memory subsystem caches conversion factors to accelerate repeated conversions:

- Save memory: Persist cache to disk (happens automatically on exit if **Use cache** enabled)
- Clean memory: Clear all cached conversion factors from RAM
- Reload memory: Refresh cache from disk file
- Export memory: Save cache to custom file location
- Import memory: Load cache from external file

Use export/import to share conversion caches between machines or backup expensive search results.

7.3.2 Delete Cache Files

Removes all **.cache** files from the Unyts user directory. Useful for troubleshooting or forcing complete rebuild.

7.4 Options Menu

7.4.1 Set FVF (Formation Volume Factor)

For petroleum engineering workflows converting between reservoir and surface volumes. Enter FVF in:

- Dimensionless form: **1.25 vol/vol**
- With units: **1.5 rb/MMscf**

Once set, enables conversions like `convert(10, 'rb', 'sm3')` using FVF correction.

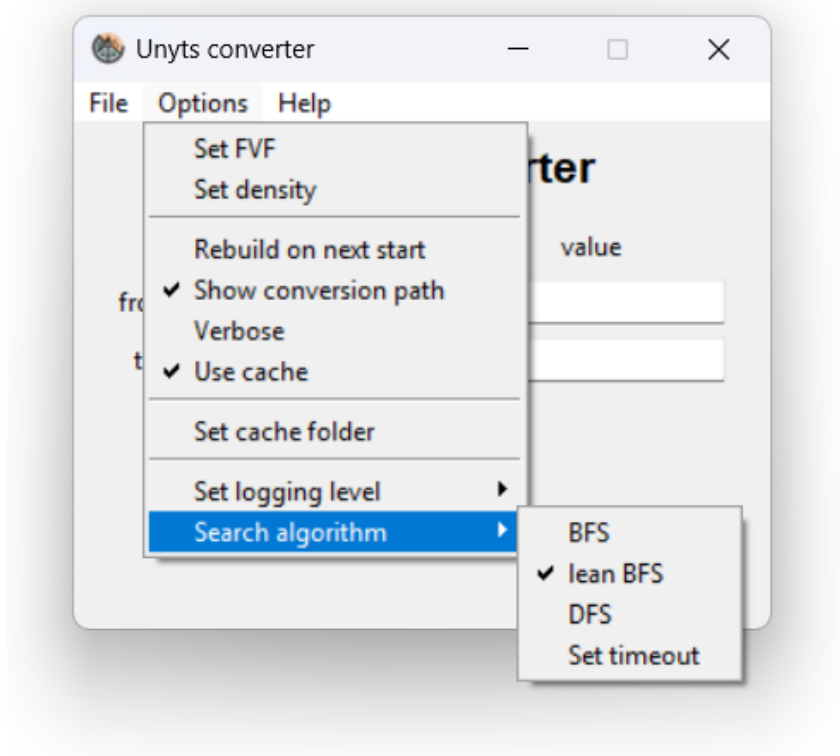


Figure 7.2: Options menu showing search algorithm selection

7.4.2 Set Density

Constant density for volume ↔ mass conversions. Enter in:

- Default: **997 g/cm³**
- With units: **8.34 lb/gal**

Enables conversions between incompatible dimensions, e.g., `convert(1, 'liter', 'kg')` using density bridge.

7.4.3 General Options

- Rebuild on next start: Force full network rebuild from database definitions (clears cache)
- Show conversion path: Display graph traversal path in marquee below button
- Verbose: Enable detailed console logging
- Use cache: Persist conversion factors across sessions
- Set cache folder: Change location where **.cache** files are stored
- Set logging level: Choose DEBUG, INFO, WARNING, ERROR, or CRITICAL verbosity

7.4.4 Search Algorithm

(Figure 7.2)

Select graph search algorithm:

- BFS: Standard breadth-first (guaranteed shortest path)
- lean BFS: Optimized BFS (default, 2.5× faster)
- hybrid BFS: Parallel BFS (experimental, requires threading/multiprocessing)
- DFS: Depth-first (faster for dense graphs, no guarantee of shortest path)
- Set timeout: Maximum search time in seconds (default 5, enter **unlimited** for no limit)

Changing algorithm automatically selects "Repeat search" to apply the new method.

7.4.5 Parallel

Enable experimental parallel search (requires Python threading or multiprocessing support):

- threading: Use Python threads (lighter weight)
- multiprocessing: Use separate processes (better CPU utilization)
- off: Disable parallelization
- auto: Automatic selection based on platform capabilities

Note: Parallel modes are currently experimental and deactivated by default on some platforms.

7.5 Help Menu

- Help: Opens Jupyter notebook demo at https://github.com/ayaranitram/unyts/blob/master/unyts_demo.ipynb
- GitHub: Links to repository at <https://github.com/ayaranitram/unyts>
- Version: Displays current Unyts version number

7.6 Conversion Path Display

When "Show conversion path" is enabled, a scrolling marquee below the button shows:

- Graph traversal path with conversion factors
- Error messages (e.g., "no conversion found", "search timeout")
- Status updates (e.g., "FVF set as 1.25 vol/vol")

Example path display:

```
[Unyts INFO]: converting from 'cm' to 'in': cm ×0.3937→ in
```

7.7 Error Handling in GUI

The GUI gracefully handles:

- `NoConversionFoundError`: Displays "no conversion found" in path marquee
- `SearchTimeoutError`: Shows "search timeout!" message
- `NoFVFError`: Prompts user to "Please set FVF constant from Options menu"
- Invalid input: Non-numeric values are ignored until valid entry

Chapter 8

Python API Reference

The Unyts Python API provides functional and object-oriented interfaces for unit conversion, arithmetic, and manipulation.

8.1 Core Functions

8.1.1 `convert()`

Primary conversion function.

Signature:

```
1 convert(value, from_unit, to_unit, algorithm='lean_BFS',  
2         parallel=None, timeout=5, verbose=False)
```

Parameters:

- **value**: Numeric (int, float, array) to convert
- **from_unit**: Source unit as string (e.g., 'cm', 'kg/m3')
- **to_unit**: Target unit as string
- **algorithm**: 'BFS', 'lean_BFS' (default), 'DFS', 'hybrid_BFS'
- **parallel**: None (auto), 't' (threading), 'p' (multiprocessing), False
- **timeout**: Maximum search time seconds (default 5)
- **verbose**: Boolean to enable logging

Returns: Converted numeric value

Raises:

- **NoConversionFoundError**: No path exists in graph
- **SearchTimeoutError**: Search exceeded timeout
- **NoFVFError**: FVF required but not set
- **WrongFormatError**: Invalid unit string format

Examples:

```

1 from unyts import convert
2
3 # Basic conversion
4 convert(100, 'cm', 'm') # → 1.0
5
6 # With NumPy array
7 import numpy as np
8 convert(np.array([1, 2, 3]), 'ft', 'in') # → array([12, 24, 36])
9
10 # Complex units
11 convert(50, 'kg/m3', 'lb/ft3') # → 3.121
12
13 # Algorithm selection
14 convert(1, 'mile', 'mm', algorithm='DFS')

```

8.1.2 units()

Factory function to instantiate appropriate Unit subclass.

Signature:

```
1 units(value, unit_str, name=None)
```

Parameters:

- **value**: Numeric quantity
- **unit_str**: Unit string (determines subclass)
- **name**: Optional descriptive label

Returns: Instance of **Length**, **Mass**, **Time**, **Energy**, **Force**, or **Temperature** subclass

Examples:

```

1 from unyts import units
2
3 length = units(50, 'cm')
4 print(length) # → 50.0 cm
5 print(type(length).__name__) # → Length
6
7 mass = units(10, 'kg', name='sample_weight')
8 print(mass.name) # → sample_weight
9
10 temp = units(25, 'C')
11 print(type(temp).__name__) # → Temperature

```

8.2 Unit Class

Base class for all unit objects. All subclasses (Length, Mass, Time, Energy, Force, Temperature, Unitless) inherit these methods.

8.2.1 Attributes

- **.value**: Numeric quantity (int, float, array)
- **.unit**: Unit string (e.g., 'm', 'kg/s2')

- `.name`: Optional descriptive string (default `None`)

Examples:

```
1 from unyts import units
2
3 distance = units(1500, 'm', name='track_length')
4 print(distance.value) # → 1500
5 print(distance.unit) # → m
6 print(distance.name) # → track_length
```

8.2.2 Arithmetic Operations

Units support standard Python operators. Results adopt units of the first operand.

Addition / Subtraction

```
1 from unyts import units
2
3 a = units(100, 'cm')
4 b = units(1, 'm')
5 result = a + b
6 print(result) # → 200.0 cm (converts m to cm, then adds)
7
8 c = units(5, 'ft')
9 d = units(12, 'in')
10 print(c - d) # → 4.0 ft (converts in to ft, then subtracts)
```

Multiplication / Division

Creates compound units:

```
1 force = units(10, 'kg') * units(9.8, 'm/s2')
2 print(force) # → 98.0 kg·m/s2 (Force)
3
4 velocity = units(100, 'm') / units(10, 's')
5 print(velocity) # → 10.0 m/s
6
7 # Dimensional analysis
8 energy = units(50, 'kg') * units(2, 'm/s')**2
9 print(energy) # → 200.0 kg·m2/s2 (Energy)
```

Exponentiation

```
1 area = units(5, 'm')**2
2 print(area) # → 25.0 m2
3
4 volume = units(3, 'ft')**3
5 print(volume) # → 27.0 ft3
```

8.2.3 Comparison Operators

Units automatically convert to common units before comparing:

```
1 from unyts import units
2
3 a = units(100, 'cm')
4 b = units(1, 'm')
5 print(a == b)      # → True (converts to same units)
6 print(a < b)       # → False
7 print(a <= b)      # → True
8
9 c = units(1, 'mile')
10 d = units(1600, 'm')
11 print(c > d)       # → True (1 mile = 1609.34 m)
```

8.2.4 .to() Method

Convert instance to different units, modifying in place:

```
1 from unyts import units
2
3 distance = units(1000, 'm')
4 distance.to('km')
5 print(distance)    # → 1.0 km
6 print(distance.value) # → 1.0
7 print(distance.unit)  # → km
8
9 # Method chaining
10 temp = units(100, 'C').to('F')
11 print(temp)       # → 212.0 °F
```

8.3 Module-Level Functions

8.3.1 Configuration

```
1 import unyts
2
3 # Algorithm selection
4 unyts.set_algorithm('BFS') # 'BFS', 'lean_BFS', 'DFS', 'hybrid_BFS'
5 current = unyts.get_algorithm()
6
7 # Parallelization
8 unyts.set_parallel('t') # 't' (threading), 'p' (multiprocessing), False, None
9 unyts.set_parallel(False) # Disable
10
11 # Search timeout
12 unyts.set_timeout(10) # seconds, None for unlimited
13
14 # Logging
15 unyts.verbose(True) # Enable verbose output
16 unyts.verbose(False) # Disable
17
18 # Network rebuild
19 unyts.reload() # Force rebuild from database
```

8.3.2 Memory Management

```
1 from unyts.database import save_memory, load_memory, clean_memory
2
3 # Cache operations
4 save_memory()           # Persist to default location
5 save_memory('custom.cache') # Export to file
6 load_memory()           # Reload from default
7 load_memory('custom.cache') # Import from file
8 clean_memory()          # Clear cache from RAM
```

8.3.3 Path Inspection

Visualize conversion paths:

```
1 import unyts
2
3 unyts.print_path(True)  # Enable path printing
4 result = unyts.convert(100, 'cm', 'in')
5 # Output: [Unyts INFO]: converting from 'cm' to 'in':
6 #           cm ×0→.3937 in
7
8 unyts.print_path(False) # Disable
```

8.3.4 Custom Units and Conversions

Add New Unit

```
1 from unyts.database import units_network
2
3 # Define unit with conversion to existing unit
4 units_network.set_unit('smoot', 1.7018, 'm')
5 # 1 smoot = 1.7018 meters
6
7 result = unyts.convert(10, 'smoot', 'ft')
8 # Now 'smoot' is available in all conversions
```

Add Direct Conversion

```
1 from unyts.database import units_network
2
3 # Add conversion factor between two existing units
4 units_network.set_conversion('furlong', 'km', 0.201168)
5 # Sets: 1 furlong = 0.201168 km
6
7 # Creates bidirectional edge in graph
8 result = unyts.convert(5, 'km', 'furlong')
```

8.3.5 Formation Volume Factor (FVF)

For petroleum engineering:


```

1 from unyts.database import units_network
2
3 # Set FVF (reservoir vol / surface vol)
4 units_network.set_fvf(1.25) # dimensionless vol/vol
5
6 # Now enables conversions:
7 result = unyts.convert(1000, 'rb', 'sm3') # reservoir barrel → surface m³

```

8.3.6 Constant Density

Bridge volume and mass:

```

1 import unyts
2
3 # Set density for volume ↔ mass conversions
4 unyts.set_density(997) # g/cm³ (water at 25°C)
5
6 # Now enables:
7 mass = unyts.convert(1, 'liter', 'kg') # → 0.997 kg
8 volume = unyts.convert(500, 'g', 'ml') # → ~500.5 ml

```

8.4 Type Checking and Subclasses

```

1 from unyts import units
2 from unyts.units import Length, Mass, Temperature
3
4 distance = units(100, 'm')
5 weight = units(50, 'kg')
6
7 # Type checking
8 isinstance(distance, Length) # → True
9 isinstance(weight, Mass) # → True
10 isinstance(weight, Length) # → False
11
12 # Direct instantiation (alternative to units())
13 temp = Temperature(25, 'C')
14 force = Force(100, 'N')

```

8.5 NumPy and Pandas Integration

Unyts seamlessly handles array-like values:

```

1 import numpy as np
2 import pandas as pd
3 from unyts import convert, units
4
5 # NumPy arrays
6 arr = np.array([1, 2, 3, 4, 5])
7 result = convert(arr, 'km', 'm')
8 print(result) # → [1000. 2000. 3000. 4000. 5000.]
9
10 # Pandas Series

```

```
11 series = pd.Series([10, 20, 30], name='distances')
12 converted = convert(series, 'mile', 'km')
13 print(converted)
14 # 0    16.09
15 # 1    32.19
16 # 2    48.28
17 # Name: distances, dtype: float64
18
19 # Unit objects with arrays
20 temps = units(np.array([0, 25, 100]), 'C')
21 temps.to('F')
22 print(temps.value) # → [ 32.  77. 212.]
```

8.6 Uncertain Units

For measurements with uncertainty:

```
1 from unyts import units
2
3 # Create uncertain unit
4 length = units("100±5", "cm", name="measurement")
5 print(length) # → (100±5) cm
6
7 # Propagates through arithmetic
8 doubled = length * 2
9 print(doubled) # → (200±10) cm
10
11 # Conversion preserves uncertainty
12 in_meters = length.to('m')
13 print(in_meters) # → (1.0±0.05) m
```

Chapter 9

Troubleshooting

9.1 Installation Issues

9.1.1 Module Not Found

Problem: `ModuleNotFoundError: No module named 'unyts'`

Solution:

1. Verify installation: `pip list | grep unyts`
2. Reinstall: `pip install --upgrade --force-reinstall unyts`
3. Check Python environment (virtual env vs system)
4. Ensure pip installs to correct Python interpreter

9.1.2 Dependency Conflicts

Problem: Package version conflicts during installation

Solution:

```
1 # Create clean virtual environment
2 python -m venv unyts_env
3 source unyts_env/bin/activate # Linux/Mac
4 unyts_env\Scripts\activate    # Windows
5
6 pip install unyts
```

9.2 Conversion Errors

9.2.1 NoConversionFoundError

Problem: `unyts.errors.NoConversionFoundError: No conversion found between 'X' and 'Y'`

Causes and Solutions:

1. Incompatible dimensions: Cannot convert length to mass without density

```
1 # Won't work: different dimensions
2 convert(10, 'kg', 'm') # X Mass ≠ Length
3
4 # Solution: set density bridge
5 unyts.set_density(1000) # kg/m³
6 convert(10, 'kg', 'liter') # ✓ Mass → Volume via density
```

2. Typo in unit name:

```
1 convert(5, 'kilogram', 'g') # ✗ Use 'kg' not 'kilogram'
2 convert(5, 'kg', 'g')      # ✓ Correct abbreviation
```

3. Custom unit not defined:

```
1 # Define custom unit first
2 from unyts.database import units_network
3 units_network.set_unit('smoot', 1.7018, 'm')
4 convert(10, 'smoot', 'ft') # ✓ Now works
```

4. Disconnected graph component: Unit exists but has no path to target

```
1 # Add conversion bridge
2 units_network.set_conversion('unit_a', 'unit_b', factor)
```

9.2.2 SearchTimeoutError

Problem: `unyts.errors.SearchTimeoutError`: Search exceeded timeout limit

Solutions:

1. Increase timeout:

```
1 unyts.set_timeout(30) # 30 seconds
2 # or
3 convert(value, from_unit, to_unit, timeout=30)
```

2. Disable timeout:

```
1 unyts.set_timeout(None) # Unlimited time
```

3. Try faster algorithm:

```
1 unyts.set_algorithm('DFS') # Often faster for complex graphs
```

4. Enable caching: Cache results to avoid repeated searches

```
1 unyts.cache(True)
2 convert(value, from_unit, to_unit) # First call: slow search
3 convert(value2, from_unit, to_unit) # Second call: instant (cached)
```

9.2.3 NoFVFEError

Problem: `unyts.errors.NoFVFEError`: FVF must be set for reservoir/surface volume conversions

Solution:

```
1 from unyts.database import units_network
2
3 # Set formation volume factor
4 units_network.set_fvf(1.25) # vol/vol
5
6 # Now works:
7 convert(1000, 'rb', 'sm3') # Reservoir barrel → surface m³
```

9.2.4 WrongFormatError

Problem: `unyts.errors.WrongFormatError: Invalid unit format`

Common causes:

- Spaces in compound units: use `'kg/m3'` not `'kg / m3'`
- Unicode issues: ensure UTF-8 encoding for symbols like `'°C'`
- Unsupported operators: use `/` or `·` for division/multiplication

9.3 GUI Issues

9.3.1 GUI Won't Launch

Problem: `python -m unyts` fails or hangs

Solutions:

1. Check Tkinter installed:

```
1 python -c "import tkinter" # Should not error
```

2. On Linux, install Tkinter:

```
1 # Ubuntu/Debian
2 sudo apt-get install python3-tk
3
4 # Fedora/RHEL
5 sudo dnf install python3-tkinter
```

3. Verify display server (Linux):

```
1 echo $DISPLAY # Should output :0 or similar
```

9.3.2 GUI Crashes on Conversion

Problem: GUI closes or freezes during conversion

Solutions:

- Disable parallel mode: Options → Parallel → off
- Reduce timeout: Options → Search algorithm → Set timeout → 5 seconds
- Clear cache: File → Delete cache files, then restart
- Check console for error messages (run from terminal)

9.4 Performance Issues

9.4.1 Slow First Conversion

Cause: First run builds units network from database definitions (typically 2-5 seconds)

Solutions:

- Enable caching: `unyts.cache(True)` — subsequent runs load from cache instantly
- Pre-build cache: Run `python -m unyts` once to generate cache
- Cache persists across sessions in user folder

9.4.2 Slow Subsequent Conversions

Causes and solutions:

1. Complex path: Many graph hops required

```
1 # Add direct conversion shortcut
2 from unyts.database import units_network
3 units_network.set_conversion('unit_a', 'unit_b', factor)
```

2. Inefficient algorithm:

```
1 unyts.set_algorithm('lean_BFS') # Fastest for most cases
2 # or
3 unyts.set_algorithm('DFS')      # Fast for dense graphs
```

3. Cache disabled:

```
1 unyts.cache(True) # Enable persistent caching
```

9.5 Cache Corruption

Symptoms: Incorrect conversion results, crashes, or “file corrupt” errors

Solution:

```
1 from unyts.database import delete_cache
2 delete_cache() # Remove all .cache files
3
4 # Or manually delete from:
5 # Windows: C:\Users\<user>\AppData\Local\unyts\
6 # Linux/Mac: ~/.unyts/
7
8 unyts.reload() # Force rebuild
```

9.6 Logging and Debugging

9.6.1 Enable Verbose Mode

See detailed search paths and timing:

```
1 unyts.verbose(True)
2 convert(100, 'cm', 'in')
3 # Output:
4 # [Unyts INFO]: converting from 'cm' to 'in': cm ×0→.3937 in
5 # [Unyts DEBUG]: Search completed in 0.0012s
```

9.6.2 Adjust Logging Level

```
1 from unyts.helpers.logger import logger
2
3 logger.set_level('DEBUG') # Most verbose
4 logger.set_level('INFO')  # Default
5 logger.set_level('WARNING') # Errors only
```

9.6.3 Inspect Network Structure

```
1 from unyts.database import units_network
2
3 # View all units
4 all_units = units_network.get_all_units()
5 print(f"Total units: {len(all_units)}")
6
7 # Check if unit exists
8 if 'smoot' in all_units:
9     print("Unit 'smoot' is defined")
10
11 # View conversion graph statistics
12 graph = units_network.get_graph()
13 print(f"Nodes: {len(graph.nodes)}, Edges: {len(graph.edges)}")
```

9.7 Getting Help

- GitHub Issues: <https://github.com/ayaranitram/unyts/issues>
- Demo Notebook: https://github.com/ayaranitram/unyts/blob/master/unyts_demo.ipynb
- Documentation: Browse source code docstrings for detailed API reference
- Email: Contact maintainer Martín Carlos Araya at martinaraya@gmail.com

When reporting issues, include:

1. Unyts version: `python -c "import unyts; print(unyts.__version__)"`
2. Python version: `python --version`
3. Operating system
4. Minimal code example reproducing the problem
5. Full error traceback